PROJECT REPORT - 2

Multi-Sensor IoT Healthcare Data Fusion

Student Name: Sandeep Kumar

Roll Number: 202310101200021

Guided By: Dr. Purnendu Pandey

Course / Program: B.Tech (CSE)

Institution: Sitare University

Submission Date: 15 April 2026

Dataset: MHEALTH Dataset (UCI)

Research Paper : Link

GitHub Repository: Link

1. Objective

The main goal of this stage is to use the dataset I prepared earlier to create and test deep learning models that can accurately predict what people are doing based on data from multiple sensors. We also want to understand how these models work, compare how well different models perform, and figure out which approach works best for data that changes over time.

2. Transition from Part 1

The data we worked on earlier is now ready to use. I cleaned it up, made sure everything was consistent, and organized it in a way that makes sense for time-based learning models. I got rid of any useless information, like empty activity classes, and made sure all the features are on the same scale. Now, the data is set up with input sequences, which are like sensor readings, and output labels, which are the activities that go with those readings. This makes it perfect for models that learn from sequences of data.

3. Model Development Approach

Human actions, such as walking or running, can't be understood with just one piece of information. They are about how things change over time, like how fast you're moving or how your body is positioned. This means simple models aren't good enough, and we need more complex models that look at sequences of data, like those used in deep learning. These models can understand patterns in data over time, which is important for making sense of human activities.

To get a better understanding of this, several different models were tried out and compared. The simplest one was a basic RNN model, which was used as a baseline. Then, more complex models were tested, including LSTM, a combination of CNN and LSTM, a combination of CNN and GRU, and a BiLSTM combined with CNN.

4. Workflow of the System

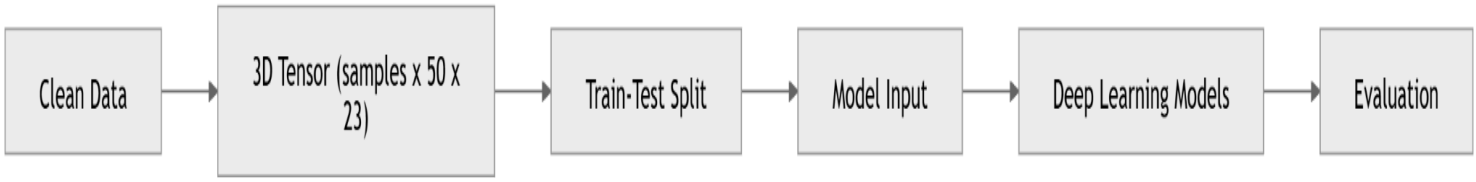

Figure 1: Workflow of the System

This system flowchart shows how everything works together, from getting clean data to making predictions and checking how well they work. Each part of the process is crucial in getting the data ready for the model to learn from and making sure the model does its job right.

5. Why Simple RNN Failed

The RNN model was our starting point, but it didn't quite live up to our expectations. One major issue with RNNs is that they struggle to hold onto information over long periods of time. As the sequence gets longer, the details from earlier on start to fade away, which makes the model less effective. This is a big problem, because it means the model can't make good use of the information it's been given, and its performance suffers as a result.

This dataset makes it tough to tell activities apart because you need to look at patterns that happen over a long time. The RNN didn't do well with this, it was inaccurate and its performance was all over the place, which means it's not a good fit for this job.

6. Model Architectures

1. **RNN:**

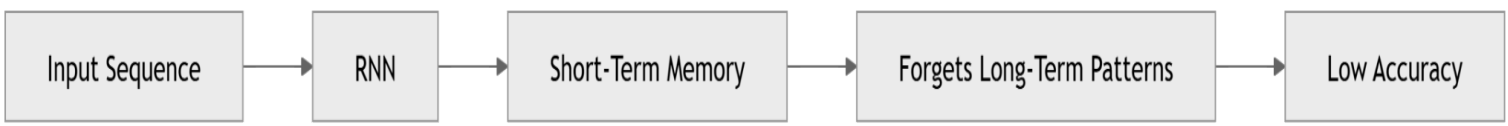

Figure 2: Model Architecture of RNN

This figure shows a simple RNN model, where it looks at input sequences one step at a time. But RNNs have a problem, they can't remember things for very long, which makes it hard for them to handle complex data that happens over time. This means they don't work as well as we want them to.

2. **LSTM:**

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50, 128)	76,800
batch_normalization (BatchNormalization)	(None, 50, 128)	512
dropout (Dropout)	(None, 50, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
batch_normalization_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout_2 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 12)	780

Total params: 131,916 (515.30 KB)
Trainable params: 131,532 (513.80 KB)
Non-trainable params: 384 (1.50 KB)

Table 1: Model Summary of LSTM

This table provides a summary of the LSTM model, including layers, output shapes, and parameters. It shows how the model processes sequential data and highlights the number of trainable parameters involved in learning.

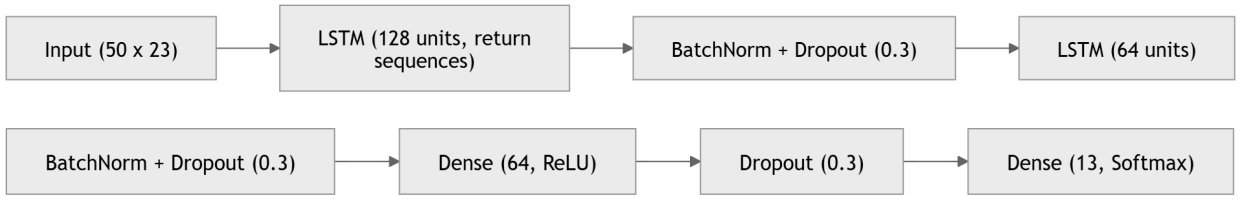

Figure 3: Model Architecture of LSTM

This diagram illustrates how LSTM processes input sequences using memory cells and gates. It helps the model retain important information over time, making it suitable for time-series data like human activity signals.

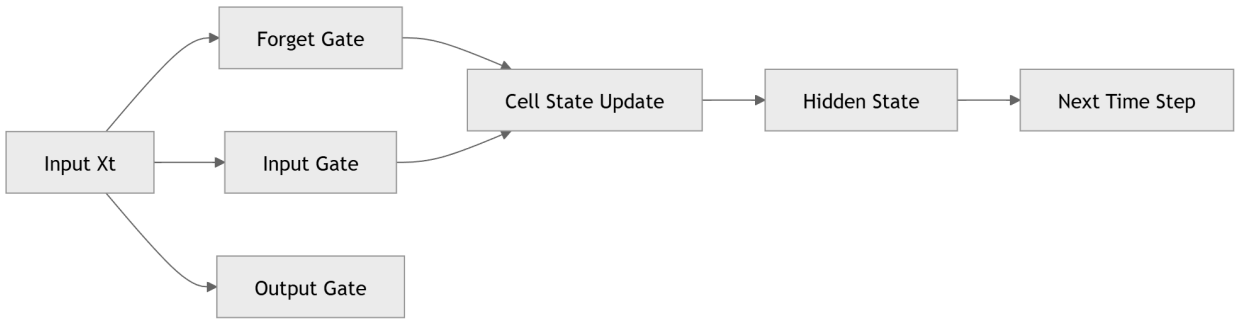

Figure 4: Architecture of LSTM

This diagram explains the internal working of LSTM, including input, forget, and output gates. These gates control how information flows through the network and allow the model to remember or forget information as needed.

3. CNN + LSTM Model:

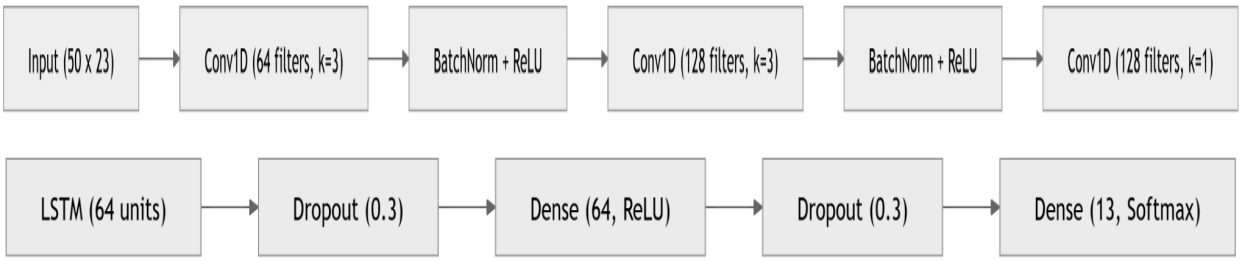

Figure 5: Model Architecture of CNN + LSTM

This diagram shows how CNN layers first extract features from the input data, and then LSTM layers learn temporal patterns from these features. This combination helps in capturing both spatial and sequential information.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv1d_7 (Conv1D)	(None, 50, 64)	4,096
batch_normalization_5 (BatchNormalization)	(None, 50, 64)	256
activation_4 (Activation)	(None, 50, 64)	0
conv1d_8 (Conv1D)	(None, 50, 128)	24,704
batch_normalization_6 (BatchNormalization)	(None, 50, 128)	512
activation_5 (Activation)	(None, 50, 128)	0
conv1d_9 (Conv1D)	(None, 50, 128)	16,512
lstm_2 (LSTM)	(None, 64)	49,408
dropout_4 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 64)	4,160
dropout_5 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 12)	780

Total params: 100,428 (392.30 KB)
Trainable params: 100,044 (390.80 KB)
Non-trainable params: 384 (1.50 KB)

Table 2: Model summary of CNN+LSTM

This table shows the structure of the CNN + LSTM model, including convolutional and sequential layers. It highlights how feature extraction and sequence learning are combined in this architecture.

4. BiLSTM + CNN Model:

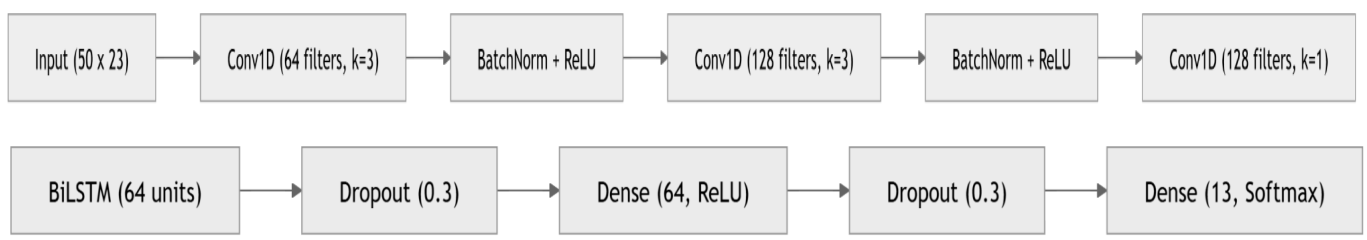

Figure 6: Model Architecture of BiLSTM + CNN

This diagram illustrates how the model processes data in both forward and backward directions using BiLSTM, followed by CNN layers. This helps in capturing more context from the data.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 50, 64)	4,096
batch_normalization (BatchNormalization)	(None, 50, 64)	256
activation (Activation)	(None, 50, 64)	0
conv1d_1 (Conv1D)	(None, 50, 128)	24,704
batch_normalization_1 (BatchNormalization)	(None, 50, 128)	512
activation_1 (Activation)	(None, 50, 128)	0
conv1d_2 (Conv1D)	(None, 50, 128)	16,512
bidirectional (Bidirectional)	(None, 128)	98,816
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 12)	780

Total params: 153,932 (601.30 KB)
Trainable params: 153,548 (599.80 KB)
Non-trainable params: 384 (1.50 KB)

Table 3: Model Summary of BiLSTM + CNN

This table summarizes the BiLSTM + CNN model, showing how bidirectional learning is applied along with feature extraction layers. It reflects the increased complexity of the model.

5. CNN + GRU Model

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 50, 64)	4,096
batch_normalization (BatchNormalization)	(None, 50, 64)	256
conv1d_1 (Conv1D)	(None, 50, 128)	24,704
batch_normalization_1 (BatchNormalization)	(None, 50, 128)	512
gru (GRU)	(None, 128)	99,072
dense (Dense)	(None, 128)	16,512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 12)	1,548

Total params: 146,700 (573.05 KB)
Trainable params: 146,316 (571.55 KB)
Non-trainable params: 384 (1.50 KB)

Table 4: Model Summary of CNN + GRU

This table presents the structure of the CNN + GRU model, including convolutional and GRU layers. It shows how GRU provides a simpler alternative to LSTM with fewer parameters.

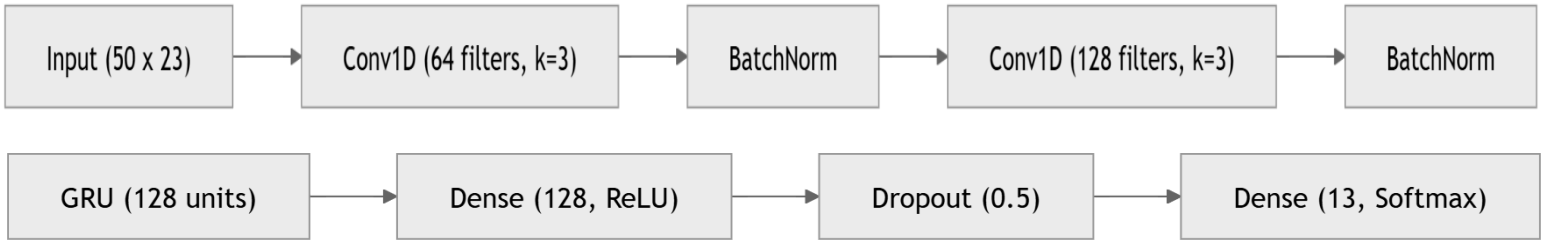

Figure 7: Model Architecture of CNN + GRU

This diagram shows how CNN extracts features and GRU processes sequential patterns. It provides a balance between performance and computational efficiency.

7. Training Process

When we train our models, we use sequences that are 50 timesteps long and have 23 different features. As the model is trained, it tries to predict what activity labels are, figures out how wrong it is using something called categorical crossentropy, and then uses backpropagation to update the weights so it can get better at predicting. This process helps the model learn from its mistakes and improve over time.

To get the best results, we used the Adam optimizer with a learning rate of 0.0005. This helped our models learn quickly and efficiently. We trained them over many epochs, stopping when their performance stopped getting better. This told us that they had figured out the patterns they were supposed to learn.

The training process follows these steps:

1. Input sequences are passed into the model
2. The model generates predictions for activity classes
3. Loss is calculated using categorical crossentropy
4. Backpropagation is applied to update weights
5. The optimizer adjusts parameters to minimize error

```
Epoch 1/40
771/771 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4558 - loss: 1.5837
Epoch 1: val_accuracy improved from None to 0.81741, saving model to /content/drive/MyDrive/Sensor_DL/cnn_gru_best.keras

Epoch 1: finished saving model to /content/drive/MyDrive/Sensor_DL/cnn_gru_best.keras
771/771 ━━━━━━━━━━━━━━━━━━━━ 19s 13ms/step - accuracy: 0.6165 - loss: 1.1128 - val_accuracy: 0.8174 - val_loss: 0.5357
Epoch 2/40
770/771 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8168 - loss: 0.5262
Epoch 2: val_accuracy improved from 0.81741 to 0.87037, saving model to /content/drive/MyDrive/Sensor_DL/cnn_gru_best.keras

Epoch 2: finished saving model to /content/drive/MyDrive/Sensor_DL/cnn_gru_best.keras
771/771 ━━━━━━━━━━━━━━━━━━━━ 8s 10ms/step - accuracy: 0.8317 - loss: 0.4804 - val_accuracy: 0.8704 - val_loss: 0.3794
Epoch 3/40
769/771 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8803 - loss: 0.3423
Epoch 3: val_accuracy improved from 0.87037 to 0.89444, saving model to /content/drive/MyDrive/Sensor_DL/cnn_gru_best.keras

Epoch 3: finished saving model to /content/drive/MyDrive/Sensor_DL/cnn_gru_best.keras
771/771 ━━━━━━━━━━━━━━━━━━━━ 9s 12ms/step - accuracy: 0.8880 - loss: 0.3201 - val_accuracy: 0.8944 - val_loss: 0.3003
```

Figure 8: Model Training Output (Epoch vs Accuracy & Loss).

This figure shows the training logs, including accuracy and loss values across epochs. It indicates how the model improves over time and stabilizes as training progresses.

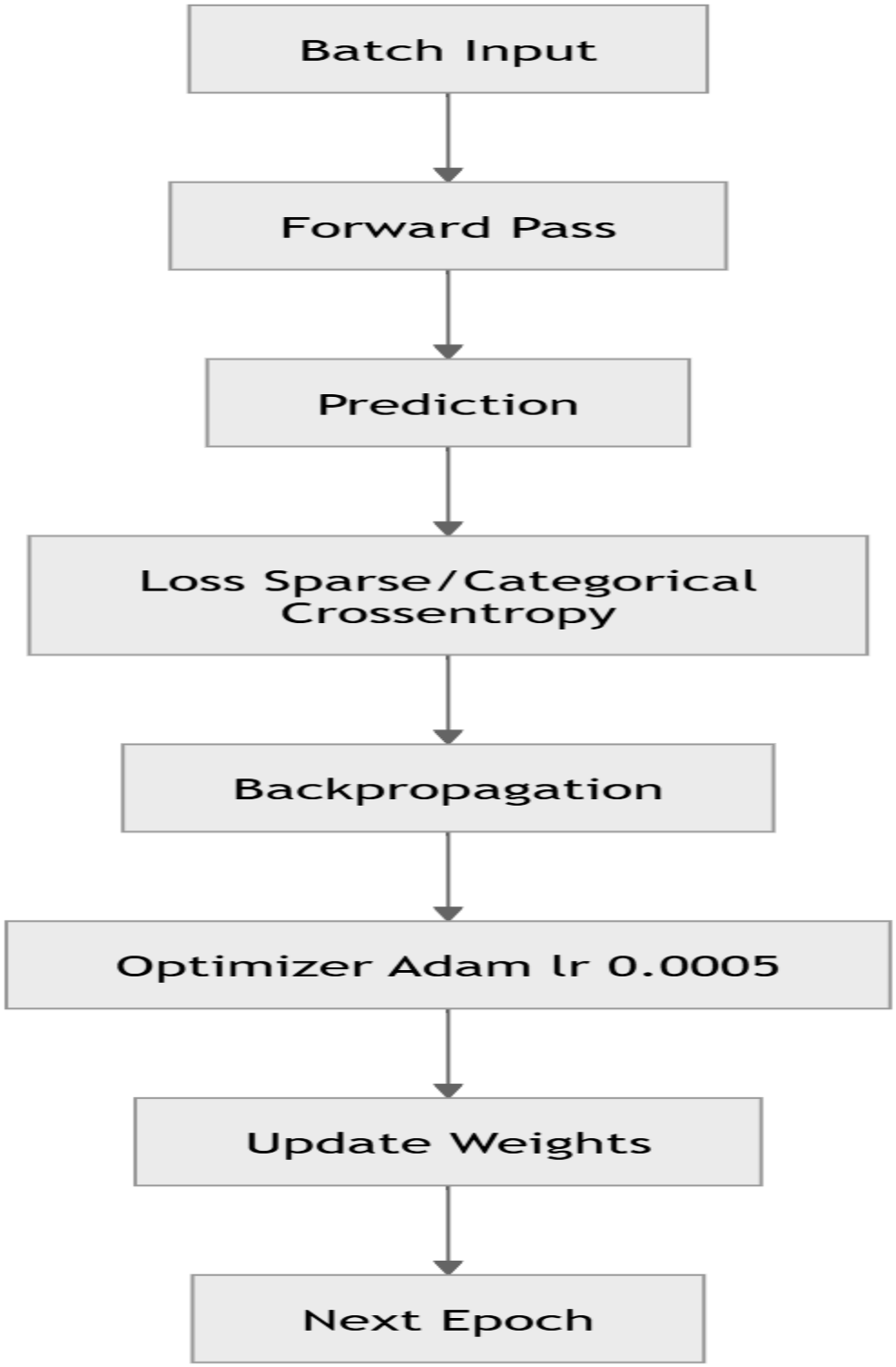

Figure 9: Training Process

This explains the step-by-step training process, starting from input sequences to prediction, loss calculation, backpropagation, and weight updates. How the model learns from data iteratively.

LSTM Model:

1. Accuracy vs Epoch (LSTM):

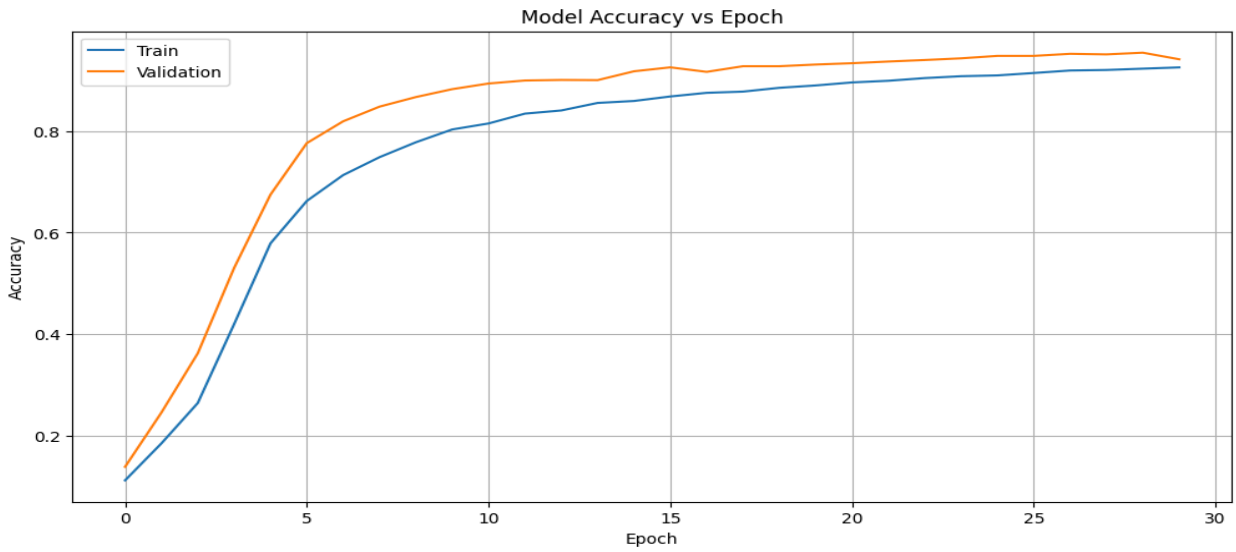

Figure 10: Accuracy vs Epoch (LSTM)

The accuracy graph shows that the model gradually improves over epochs and stabilizes after around 20 epochs, indicating effective learning. The validation accuracy remains slightly higher than the training accuracy, which suggests that the model generalizes well and does not suffer from overfitting.

2. Loss vs Epoch (LSTM):

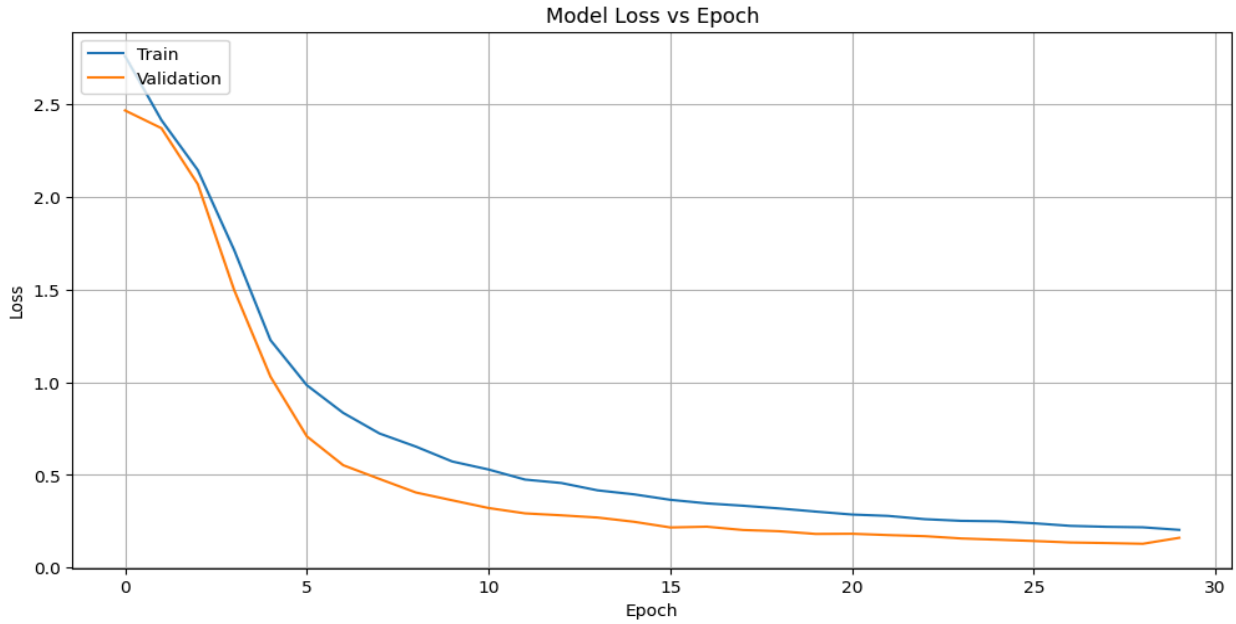

Figure 11: Loss vs Epoch (LSTM)

The loss curve decreases steadily for both training and validation sets, confirming that the model is successfully minimizing prediction error. The absence of divergence between training and validation loss indicates stable training and good model performance.

CNN + LSTM Model :

1. Accuracy vs Epoch (CNN + LSTM):

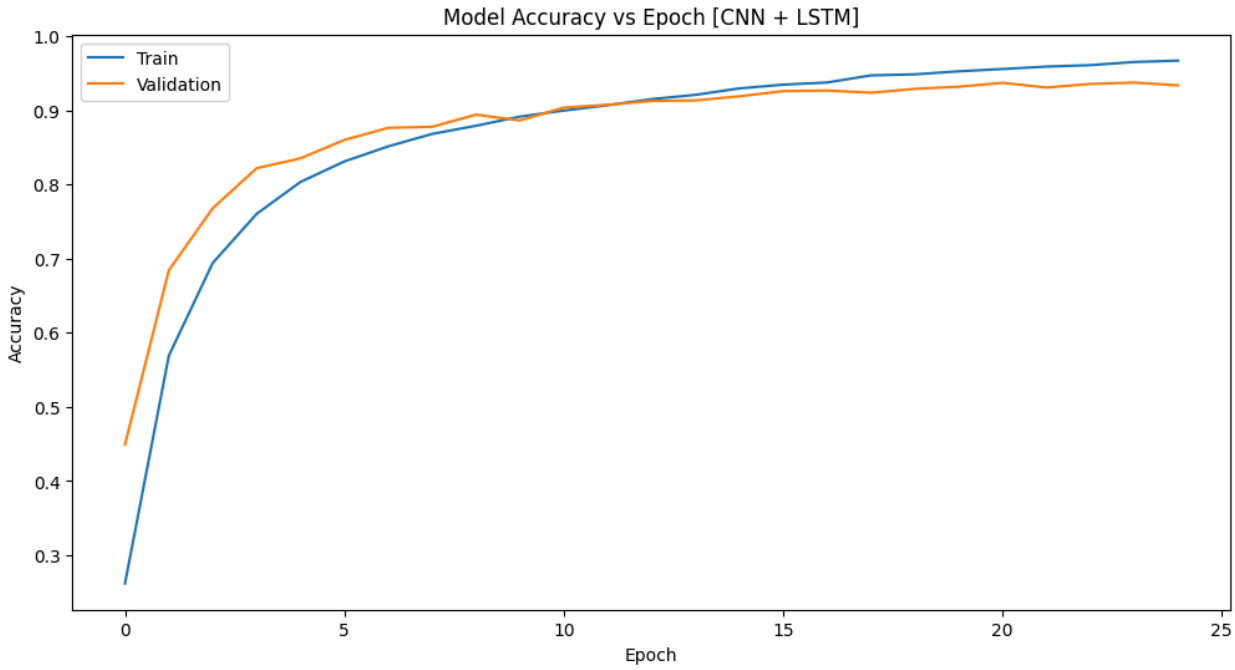

Figure 12: Accuracy vs Epoch (CNN + LSTM)

The CNN + LSTM model shows a rapid increase in accuracy during early epochs due to effective feature extraction by convolutional layers. However, a gap between training and validation accuracy appears in later epochs, indicating slight overfitting.

2. Loss vs Epoch (CNN + LSTM):

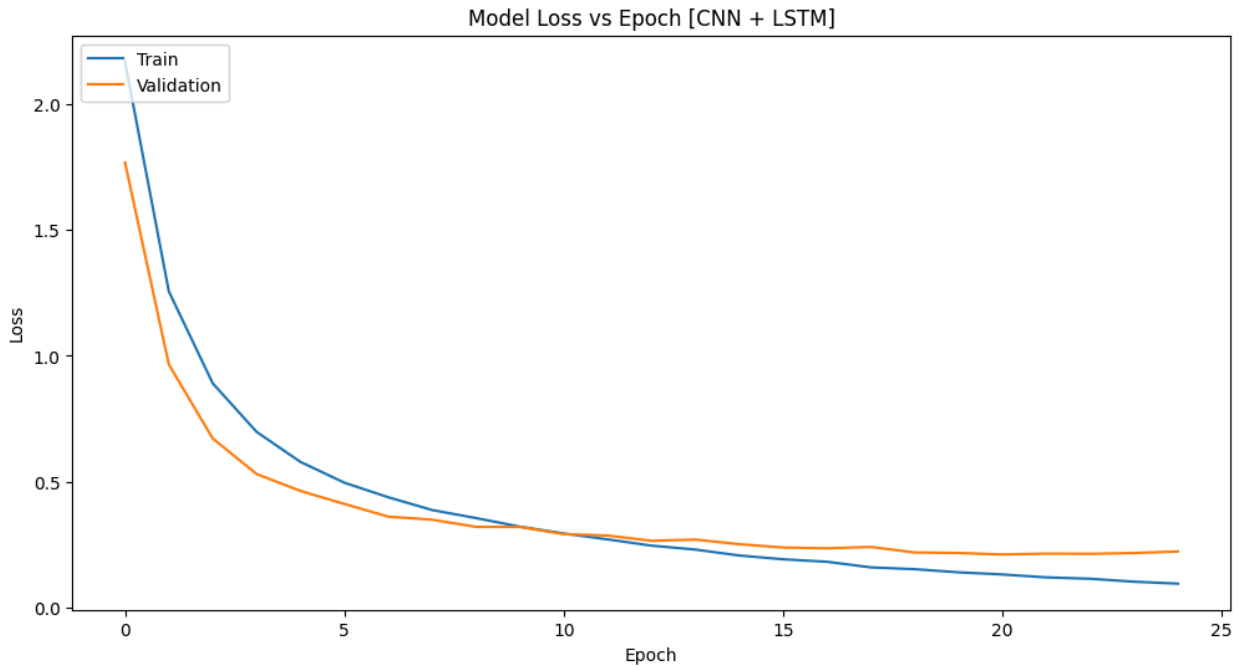

Figure 13: Loss vs Epoch (CNN + LSTM)

The loss curve shows a steady decrease in training loss, while validation loss stabilizes after a certain point. This suggests that the model starts overfitting and does not generalize as well as the LSTM model.

Conclusion the Comparison of Models:

Compared to the LSTM model, the CNN + LSTM model shows slightly higher training performance but lower generalization capability, indicating that temporal learning is more important than feature extraction for this dataset.

8. Implementation Details

The models were implemented using Python with deep learning libraries.

1. Programming Language: Python
2. Libraries Used: TensorFlow/Keras, NumPy, Pandas, Matplotlib
3. Input Shape: (50 timesteps, 23 features)
4. Output Classes: 13 activities
5. Optimizer: Adam
6. Learning Rate: 0.0005
7. Loss Function: Categorical Crossentropy
8. Batch Size: 32
9. Epochs: 30–50 (depending on model)
10. Platform: Google Colab with GPU

The dataset was first converted into sequences of fixed length (50), which allowed the models to learn temporal dependencies.

9. Results and Observations

After training different deep learning models, their performance was evaluated on the test dataset.

1. Model Performance Comparison

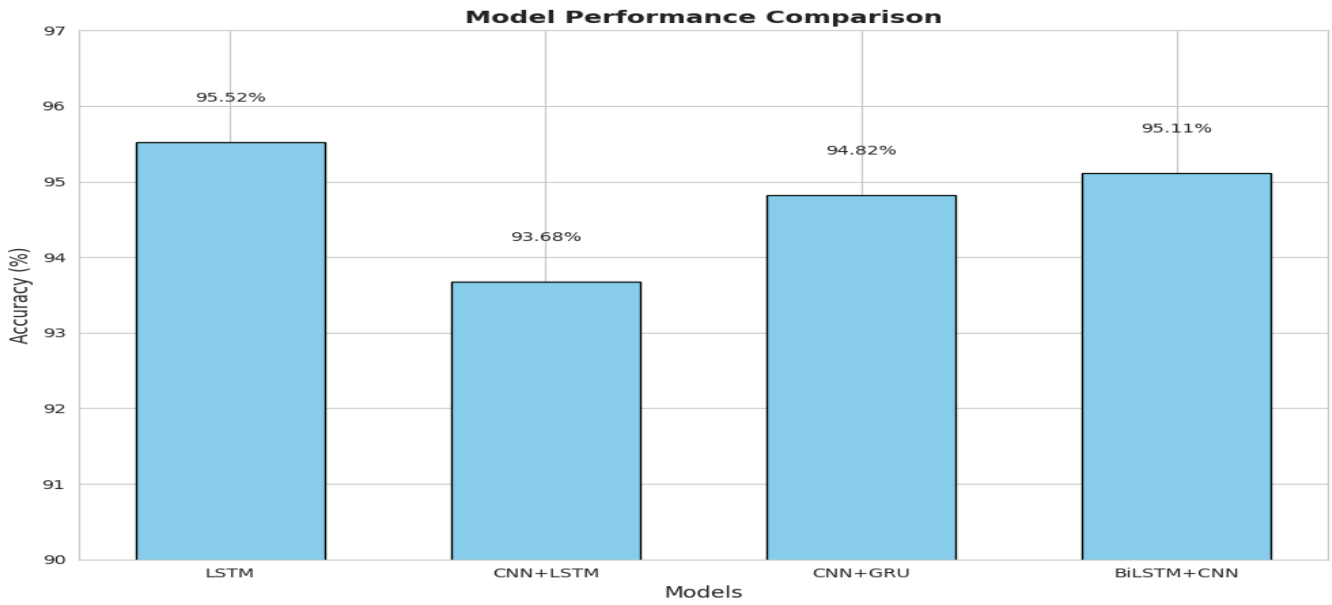

Figure 14: Model Performance Comparison

This bar graph compares the accuracy of different models. It clearly shows that the LSTM model performs best, followed by BiLSTM + CNN and CNN + GRU, while CNN + LSTM performs slightly lower. The LSTM model achieved the highest accuracy (95.52%), demonstrating strong capability in capturing temporal dependencies. The BiLSTM + CNN model also performed well (95.11%), benefiting from bidirectional learning. CNN + GRU achieved balanced performance (94.82%), while CNN + LSTM showed slightly lower accuracy (93.68%).
It seems that models which focus mainly on learning from time-related data do better than models that combine different approaches for this particular set of data.

2. Confusion Matrix

LSTM:

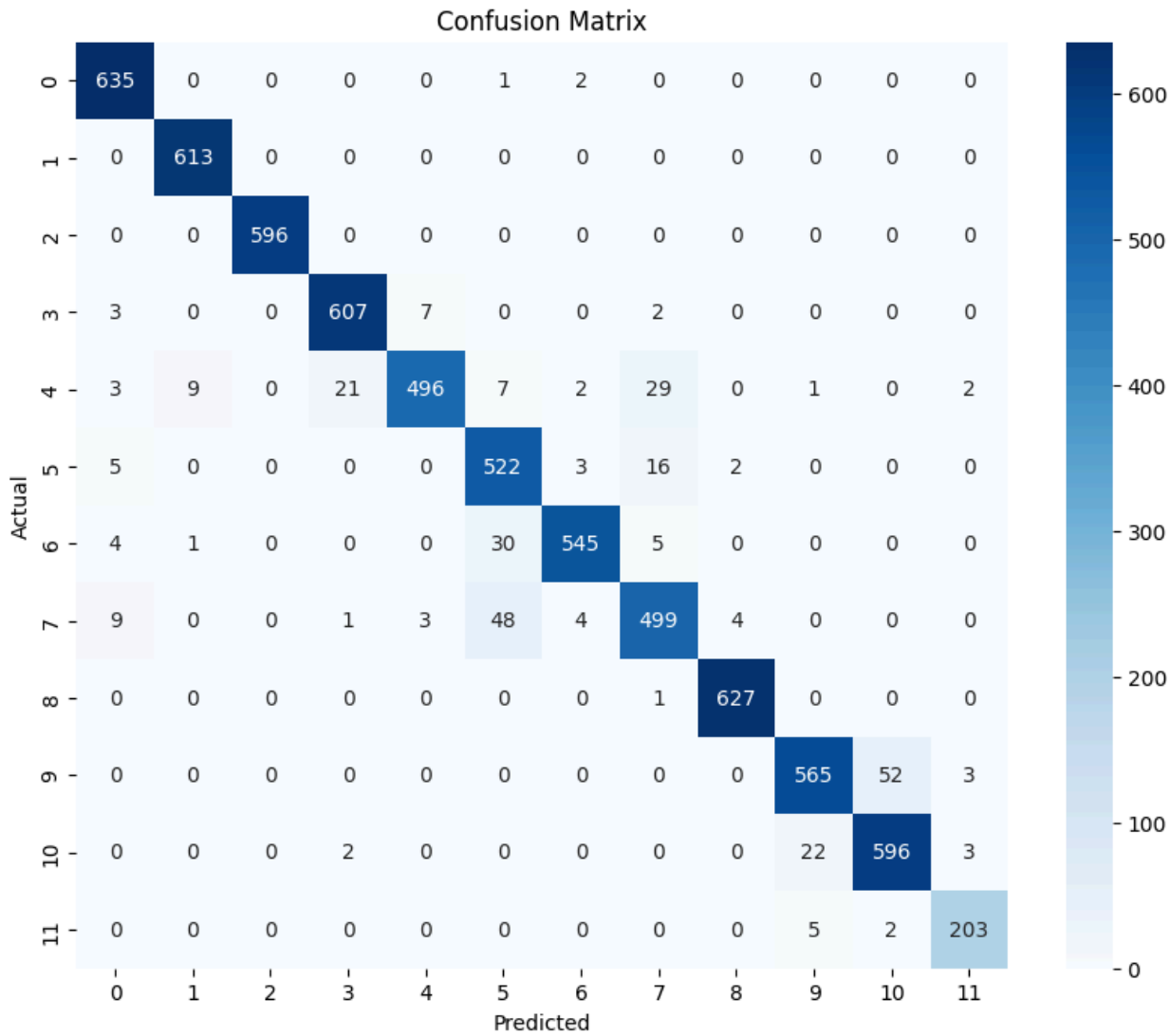

Figure 15: Confusion Matrix of LSTM Model

This matrix shows how well the model predicts each activity. Most values lie along the diagonal, indicating correct predictions. Some misclassifications occur between similar activities, which is expected in real-world data.

Overall Performance Summary:

Aspect	Observation
Correct Predictions	Very high (strong diagonal values)
Misclassifications	Very low
Model Accuracy	High (95%)
Generalization	Strong

Table 5: LSTM Confusion matrix performance summary

This table summarizes the overall performance of the model based on the confusion matrix. It highlights that the model achieves high accuracy with very few misclassifications.

Confusion Between Similar Activities:

Actual Class	Confused With	Reason
Class 4	Class 3, 7	Similar motion patterns
Class 7	Class 5	Overlapping dynamic activity
Class 9	Class 10	Nearly identical behavior

Table 6: Confusion between similar Activities

This table identifies specific activity pairs where the model gets confused. These errors occur because some activities have very similar sensor patterns.

3. Evaluation Metrics:

Model	Accuracy (%)	Precision (Weighted)	Recall (Weighted)	F1-Score (Weighted)
LSTM	**95.52**	**0.96**	**0.95**	**0.95**
CNN + BiLSTM	95.11	0.96	0.96	0.96
CNN + GRU	94.82	0.95	0.95	0.95
CNN + LSTM	93.88	0.9392	0.9388	0.9388

Table 7: Comparison of evaluation metrics across different deep learning models

If we look at the numbers, we can see how different models stack up in terms of accuracy, precision, recall, and F1-score. What really stands out is that the LSTM model does a great job overall, which isn't surprising given its reputation for handling time-series data well. This suggests that LSTM is a good choice for this type of task, and its performance confirms that.

- Accuracy: Measures how many predictions are correct
- Precision: Measures how many predicted activities are actually correct
- Recall: Measures how many actual activities are correctly identified
- F1-Score: Balance between precision and recall

10. Key Learning

Working with this project really helped us understand how deep learning models work with time-series data. We found out that LSTM is actually better than other models because it can catch long-term dependencies.

One important thing we learned is that making the model more complex doesn't always make it work better. What's more important is choosing the right model for the type of data you're working with. This is a key point because it can really affect how well your model performs. By understanding this, we can make better decisions when it comes to selecting models for our data.

We also ran into some challenges, like dealing with huge datasets and figuring out the right hyperparameters, but it actually helped us get better at solving real-world problems.

11. Challenges Faced

During this phase of the project, several challenges were encountered. One of the main issues was the large size of the dataset, which made training time-consuming. Handling such a large amount of data required careful preprocessing and efficient use of resources.

One of the tough parts was that some activities looked pretty similar. For example, sitting and standing, or walking and climbing stairs, would often produce sensor patterns that were really alike, which made it hard for our model to tell them apart. This was a challenge because the model had to be able to pick up on the tiny differences between these activities in order to work properly.

Additionally, tuning the model parameters required multiple experiments. Finding the right combination of hyperparameters was important to achieve good performance.

12. Conclusion

This research shows that models using LSTM are really good at recognizing human activity when using data from multiple sensors. The findings prove that understanding patterns over time is vital to solving this problem. Overall, the project stresses the importance of choosing the right models based on the characteristics of the data and demonstrates that learning from sequences is key to building systems that can make accurate predictions. It's all about picking the right tools for the job and using them to understand how things change over time, which is crucial for making good predictions.